アイリスアウトで学ぶHLSL

もうこんなのこりごりだぁ～

登壇:alion

自己紹介

HN: αlion, LandEater(旧), ふくらP

趣味: ガンダム TCG メカ ロボット

トレカは現在5タイトルくらいやってます



役職:プログラマ

最近のトピック

#白衣 #ガンダム #Blenderできるようになりたい #時間ない
#鍋は神 #時間ない #抹茶 #梅干し #Dr. pepper #バトオペ3
#時間ない #初音ミクのmetalbuild欲しい #AC6やる時間欲しい
#メカ好き #逆関節って叡智だと思う #理論よりロマン #時間ない
ゴロリ

Agenda

- 自己紹介
- 今回作ったもの
- ShaderとHLSLとは
- HLSLでの実装(Unity)
- Unityとの接続
- その他

今回作ったもの



- Shaderとは

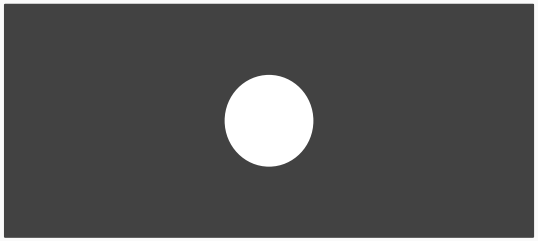
CPUではなく、GPU上で実行されて描画処理を行うプログラムのこと。

shaderには以下の3種類があります。(※読む必要はないです)

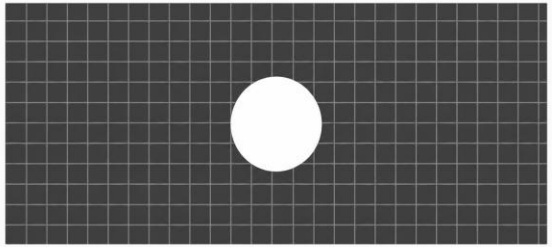
- Vertex Shader (バーテックスシェーダー)
  頂点計算とか色々やってくれるshader
- Fragment Shader（フラグメントシェーダー）
  描画対象の各フラグメントに対して、色や透明度などを計算するShader。Fragment Shaderでは、例えば以下のような計算ができる。
  テクスチャから色を取得する
  光の当たり方に応じて色を変える
  透明度を決める
  特定の領域を描画しないようにする
  今回のアイリスシェーダーでは、まさにこのFragment Shaderを利用した。
- Compute Shader（コンピュートシェーダー）
  GPUの計算資源を描画以外で使うやつ

画像を使ってやるのと何が違うのか

- 同じ表現するなら右みたいな
  画像作って移動/変形させてもいい
  →でも穴の形を変えたくなったら？
  →解像度が変わったら？



- 今回は画面全体を覆うImageにShaderを
  適用し、画面上の各位置について黒か透
  明かを計算している



- HLSLとは

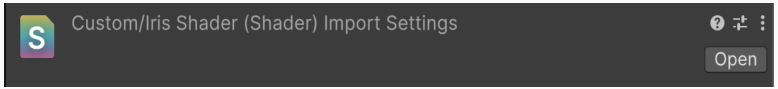
Microsoftが開発したShader(DirectX)向け言語。

float2,float3, half4などの型がある。

ベクトルがベクトルのまま計算できる

HLSLでの実装(Unity)

Create → Shader → URP Unlit Shader からShaderを作成



これをエディタで開くと右のようなコードが出ます。

重要なのは
HLSLPROGRAMとENDHLSLの間
frag関数

```
Shader "Custom/test"
{
    Properties
    {
        [MainColor] _BaseColor("Base Color", Color) = (1, 1, 1, 1)
        [MainTexture] _BaseMap("Base Map", 2D) = "white" {}
    }

    SubShader
    {
        Tags { "RenderType" = "Opaque" "RenderPipeline" = "UniversalPipeline" }

        Pass
        {
            HLSLPROGRAM

            #pragma vertex vert
            #pragma fragment frag

            #include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.hlsl"

            struct Attributes
            {
                float4 positionOS : POSITION;
                float2 uv : TEXCOORD0;
            };

            struct Varyings
            {
                float4 positionHCS : SV_POSITION;
                float2 uv : TEXCOORD0;
            };

            TEXTURE2D(_BaseMap);
            SAMPLER(sampler_BaseMap);

            CBUFFER_START(UnityPerMaterial)
                half4 _BaseColor;
                float4 _BaseMap_ST;
            CBUFFER_END

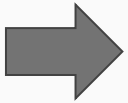
            Varyings vert(Attributes IN)
            {
                Varyings OUT;
                OUT.positionHCS = TransformObjectToHClip(IN.positionOS.xyz);
                OUT.uv = TRANSFORM_TEX(IN.uv, _BaseMap);
                return OUT;
            }

            half4 frag(Varyings IN) : SV_Target
            {
                half4 color = SAMPLE_TEXTURE2D(_BaseMap, sampler_BaseMap, IN.uv) * _BaseColor;
                return color;
            }
            ENDHLSL
        }
    }
}
```

- frag関数(fragment関数)とは

フラグメントシェーダーでどう描画するかを記述。
こいつはGPUの描画パイプラインに自動で呼ばれる。

```
half4 frag(Varyings IN) : SV_Target
{
   half4 color = SAMPLE_TEXTURE2D(_BaseMap,
sampler_BaseMap, IN.uv) * _BaseColor;
   return color;
}
```



```
half4 frag(Varyings IN) : SV_Target
{

   float2 center = _Center.xy;

   float2 centerPos = IN.uv - center;

   float aspect = _ScreenParams.x /
_ScreenParams.y;

   centerPos.x *= aspect;

   float distanceFromCamera =
length(centerPos);
   half4 color = half4(0, 0, 0, 1);

   if (distanceFromCamera < _Radius)
   {
       color = half4(0, 0, 0, 0);
   }

   return color;
}
```

- 変数の定義の仕方

HLSL内で使う変数はCBUFFER_STARTとCBUFFER_ENDの間に

```
CBUFFER_START(UnityPerMaterial)
   half4 _BaseColor;
   float4 _BaseMap_ST;
   float4 _Center;
   float _Radius;
CBUFFER_END
```

Material(後述)で保持したい変数はPropertiesに定義

```
Properties{
   [MainColor] _BaseColor("Base Color", Color) =
(1, 1, 1, 1)
   [MainTexture] _BaseMap("Base Map", 2D) =
"white" {}

   _Center("Center", Vector) = (0.5, 0.5, 0, 0)
   _Radius("Radius", Float) = 0.2
}
```

- 描画方法の変更

```
Tags { "RenderType" = "Transparent" "Queue" =
"Transparent" "RenderPipeline" =
"UniversalPipeline" }

Blend SrcAlpha OneMinusSrcAlpha
Zwrite Off
```

Unityとの接続(超簡単解説)

1. マテリアルを作成、inspectorからshaderの欄に今作成したshaderをアタッチ
2. 今回はCanvas上のimageにそのマテリアルを適用(画面を覆うように)
3. このmaterialのinspectorに表示されている値(Propertyで追加したやつ)を外部から操作して見た目を変更します
4. 例(雑)

```
private IEnumerator AnimateRadius(float targetRadius)
{
   float startRadius = _irisMaterial.GetFloat("_Radius");
   float elapsedTime = 0f;
   while (elapsedTime < _duration)
   {
       elapsedTime += Time.deltaTime;
       float t = Mathf.Clamp01(elapsedTime / _duration);
       float currentRadius = Mathf.Lerp(
           startRadius,
           targetRadius,
           t
       );
       _irisMaterial.SetFloat("_Radius", currentRadius);
       yield return null;
   }
   _irisMaterial.SetFloat("_Radius", targetRadius);
   irisCoroutine = null;
}
```

今回作ったもの



その他

